Database System
[ECEG-4172]

Chapter Four:
Relational Algebra

Outline

- Fundamental Operations of Relational Algebra
- Extended Operations
- Introduction to Relational Calculus

Relational Algebra

- ***Relational Algebra*** is a procedural query language that consists of a set of operations that take one or two relations as input and produce a new relation as a result.
- Fundamental Operations of Relational Algebra
  - ***Unary Operators:-operate on one relation***
    - Selection :- σ
    - Projection :- Π
    - Rename :- ρ
  - ***Binary Operators:- operate on pairs of relations***
    - Product (Cartesian Product) :- $\times$
    - Union :- $\cup$
    - Difference :- $-$

Unary Operations

Select Operation

- The select operation selects a subset of **tuples** from a relation instance that satisfies a given predicate (condition).

$$\sigma_C(R)$$

  - σ(sigma) represents the SELECT operator,
  - C is a Boolean expression of the select condition, and
  - R is the relation or relational algebra expression.

Example:

- From the "EMPLOYEES"relation to extract Senior Mangers, selection operation can be written as
  - Employees(EmpId, Name, BDate, Age, Gender, Position, Salary)

$$\sigma_{Position="Senior\ Manger"}(Employees)$$

- To extract employees with salary greater or equal to 3000.

$$\sigma_{Salary>=3000}(Employees)$$

Cont'd...

- **Project Operation**
  - Projection operation forms a new relation by picking **certain columns** in the relation.

    $\Pi_A(R)$

    - Π(pi) represents the PROJECT operator,
    - A is a set of attributes in the relation R , and
    - R is the relation or relational algebra expression.

Example:

- To extract Employees Name and Position only from the “EMPLOYEES”relation

$$\Pi_{\text{Name, Position}}\left(Employees\right)$$

- To extract Employees Name and Position only from the “EMPLOYEES”relation and with salary greater or equal to 3000.

$$\Pi_{\text{Name, Position}}\left(\sigma_{Salary>=3000}\left(Employees\right)\right)$$

Cont'd...

- **Rename Operation**
  - The renaming operator can be used to explicitly rename resulting relations of an expression.

    $\rho_{S(A1, A2, \ldots An)}(R)$

    - ρ represents the RENAME operator,
    - S is a name for the new relation , and
    - A1, A2, … An are new names for the attributes in the relation R.

Binary Operations

- **Cartesian Product Operation**
  - The Cartesian product operation (also known as Cross Product or Cross Join or Product) is binary set operation that generates a new relation from two relation in a combinatorial fashion.

    $R \times S$

    - × represents the PRODUCT operator, and
    - R and S are relations to be joined..

Example:

- Consider the following relations R and S, then $R \times S$ is given as shown to the right.

R	
A	B
1	2
3	4

S		
B	C	D
2	a	x
4	b	y
5	c	z

R ×S				
A	R.B	S.B	C	D
1	2	2	a	x
1	2	4	b	y
1	2	5	c	z
3	4	2	a	x
3	4	4	b	y
3	4	5	c	Z

...Example:

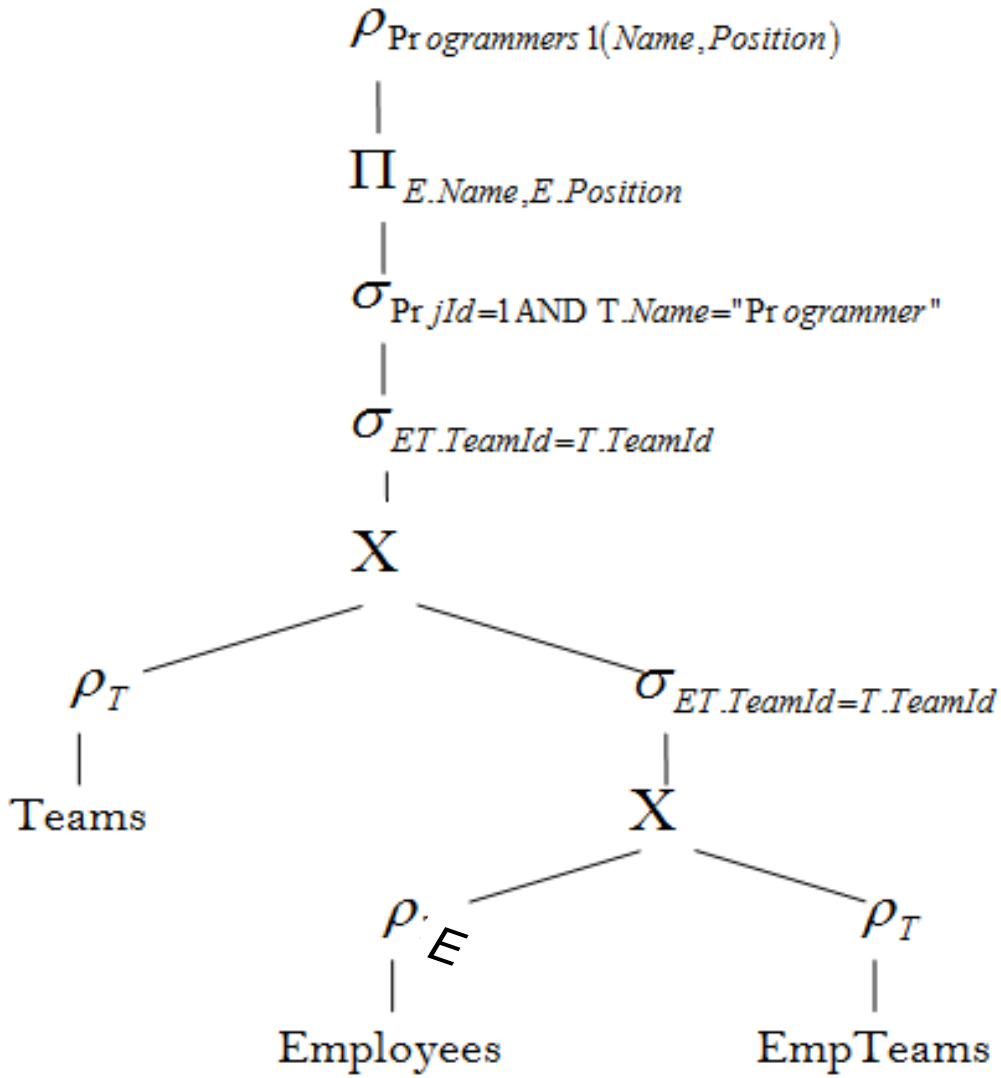
- Consider the Employees, EmpTeams and Teams relation and develop a relational algebra expression that retrieves the name and position of Employees that work on Project 1 as Programmers and rename the relation as Programmers1.
  - Employees(EmpId, Name, BDate, Age, Gender, Position, Salary)
  - EmpTeams(EmpId, TeamId)
  - Teams(TeamId, PrjId, Name, Descr)

…Example:

$$\rho_{Programmers1(\text{Name, Position})}(\Pi_{\text{E.Name, E.Position}}(\sigma_{\text{PrjId=1 AND } T.Name=\text{"Programmer"}}($$
$$\sigma_{\text{ET.TeamId=T.TeamId}}(\rho_T(Teams) \text{X} \sigma_{\text{E.EmpId=}E\text{T.EmpId}}($$
$$\rho_E(Employees) \text{X} \rho_T(EmpTeams))))))$$

- The expression tree for the above relational algebra expression is

…Example:



Expression tree for a relational algebra expression

. . . Binary Operations

- **Set Operations**
  - **Union Operation**
    - The union operation on R and S denoted by $R \cup S$ results a relation that includes all tuples either in R or in S or in both. Duplicates are eliminated from the result.
  - **Intersection Operation**
    - The intersection operation on R and S denoted by $R \cap S$ results a relation that includes all tuples in both R and S.
  - **Set Difference Operation**
    - The result of the set difference operation on R and S denoted by $R - S$ is the set of elements in R but not in S.

…Binary Operations

- For the set operations (Union, Intersection, Set difference) the two relational operands R and S must have same type of tuples, this condition is known as ***Union Compatibility***.
- Two relations $R(A_1, A_2, \ldots A_n)$ and $S(B_1, B_2, \ldots B_n)$ are said to be union compatible if
  1. They have same degree n, and
  2. $Domain(A_i) = Domain(B_i)$ for all $i = 1, 2, \ldots n$

Example:

- Consider the following relations R and S, then and R – S are given as shown to the right.

$R \cup S$

R	
A	B
1	2
3	4

S	
C	D
1	2
4	3
5	6

$R \cup S$	
A	B
1	2
3	4
4	3
5	6

R – S	
A	B
3	4

. . . Example:

(a) **STUDENT**

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

(b)

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Francis	Johnson

(c)

Fn	Ln
Susan	Yao
Ramesh	Shah

(d)

Fn	Ln
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

(e)

Fname	Lname
John	Smith
Ricardo	Browne
Francis	Johnson

Figure 6.4
The set operations UNION, INTERSECTION, and MINUS. (a) Two union-compatible relations. (b) STUDENT ∪ INSTRUCTOR. (c) STUDENT ∩ INSTRUCTOR. (d) STUDENT − INSTRUCTOR. (e) INSTRUCTOR − STUDENT.

…Example:

- Find name and position of Employees that work on both Projects 1 and 2 as Programmers.
- Similar to the previous example, we can have

$$\rho_{Programmers\,2(\text{Name, Position})}(\Pi_{\text{E.Name, E.Position}}(\sigma_{\text{PrjId=2 AND T.Name="Programmer"}}($$
$$\sigma_{\text{ET.TeamId=T.TeamId}}(\sigma_{\text{E.EmpId=}E\text{T.EmpId}}($$
$$\rho_E(Employees)\text{X}\rho_T(EmpTeams))\text{X}\rho_T(Teams)))))$$

- The employees working in both projects 1 and 2 are then given by relational algebra:

$$Programmers1 \cap Programmers2$$

More Operations

- The set of relational algebra operations $\{\sigma, \Pi, \rho, \times, \cup, -\}$ is a complete set.
- **Additional Operations**
  - Natural Join Operation
  - Theta Join Operation
  - Assignment Operation
  - Division Operation
- **Extended Operations**
  - Outer Join Operation
  - Extended Projection Operation
  - Duplicate Elimination
  - Aggregation and Grouping Operation
  - Sorting Operation

Additional Operations

- **Natural Join Operation**
  - Natural Join is a special product operation that joins two relations by:
    - **Equating** attributes of the **same name**, and
    - Projecting out **one copy** of each pair of **equated attributes**.

    $R \bowtie S$

    - $\bowtie$ represents the NATURAL JOIN operator, and
    - R and S are relations to be joined.

Example:

- Consider the following relations R and S, then R⋈ S is given as shown to the right.

R

A	B
1	2
3	4

S

B	C	D
2	a	x
4	b	y
5	c	z

R ▷◁S

A	B	C	D
1	2	a	x
3	4	b	y

Exersice:

- Consider the following relations U and V, then $U \bowtie V$?

U		
A	B	C
1	2	3
6	7	8
9	7	8

V		
B	C	D
2	3	4
2	3	5
7	8	10

Solution:

- Consider the following relations U and V, then $U \bowtie V$ is given as shown to the right.

U		
A	B	C
1	2	3
6	7	8
9	7	8

V		
B	C	D
2	3	4
2	3	5
7	8	10

$U \triangleright\triangleleft V$			
A	B	C	D
1	2	3	4
1	2	3	5
6	7	8	10
9	7	8	10

…Example:

- The pervious example that retrieves the name and position of Employees that work on Project 1 as Programmers from the modified relations below can be simplified as:
  - Employees(<u>EmpId</u>, FullName, BDate, Age, Gender, Position, Salary)
  - EmpTeams(<u>EmpId</u>, <u>TeamId</u>)
  - Teams(<u>TeamId</u>, PrjId, TName, Descr)

$$\Pi_{\text{E.FullName, E.Position}}(\sigma_{\text{T.PrjId=1 AND T.TName="Programmer"}}((\rho_E(Employees) \bowtie \rho_T(EmpTeams)) \bowtie \rho_T(Teams)))$$

…Additional Operations

- **Theta Join Operation**
  - A Theta Join is a join operation that connects relations to an arbitrary condition C.

    $R \underset{C}{\bowtie} S$
  - The result of the theta operation is constructed by:
    - Taking the product of R and S, and
    - Selecting only those tuples satisfying the condition C.

Example:

- Consider the following relations R and S, then $R \underset{\text{R.B<S.B}}{\bowtie} S$ is given as shown to the right.

R	
A	B
1	2
3	4

S		
B	C	D
2	a	x
4	b	y
5	c	z

$R \underset{\text{R.B<S.B}}{\bowtie} S$

A	R.B	S.B	C	D
1	2	4	b	y
1	2	5	c	z
3	4	5	c	Z

Exercise:

- Consider the following relations U and V, then $U \underset{A<D}{\bowtie} V$?

U		
A	B	C
1	2	3
6	7	8
9	7	8

V		
B	C	D
2	3	4
2	3	5
7	8	10

Solution:

- Consider the following relations U and V, then $U \underset{A<D}{\bowtie} V$ is given as shown to the right.

U

A	B	C
1	2	3
6	7	8
9	7	8

V

B	C	D
2	3	4
2	3	5
7	8	10

$U \underset{A<D}{\bowtie} V$

A	U.B	U.C	V.B	V.C	D
1	2	3	2	3	4
1	2	3	2	3	5
1	2	3	7	8	10
6	7	8	7	8	10
9	7	8	7	8	10

Exercise:

- For the same relations U and V, find the theta relation that has a more complex condition:

$$U \underset{A<D \text{ AND } U.B \neq V.B}{\bowtie} V$$

U

A	B	C
1	2	3
6	7	8
9	7	8

V

B	C	D
2	3	4
2	3	5
7	8	10

$U \underset{A<D}{\bowtie} V$

A	U.B	U.C	V.B	V.C	D
1	2	3	2	3	4
1	2	3	2	3	5
1	2	3	7	8	10
6	7	8	7	8	10
9	7	8	7	8	10

Solution:

- For the same relations U and V, the theta relation that has a more complex condition:

$$U \underset{A<D \text{ AND } U.B \neq V.B}{\bowtie} V$$

U		
A	B	C
1	2	3
6	7	8
9	7	8

V		
B	C	D
2	3	4
2	3	5
7	8	10

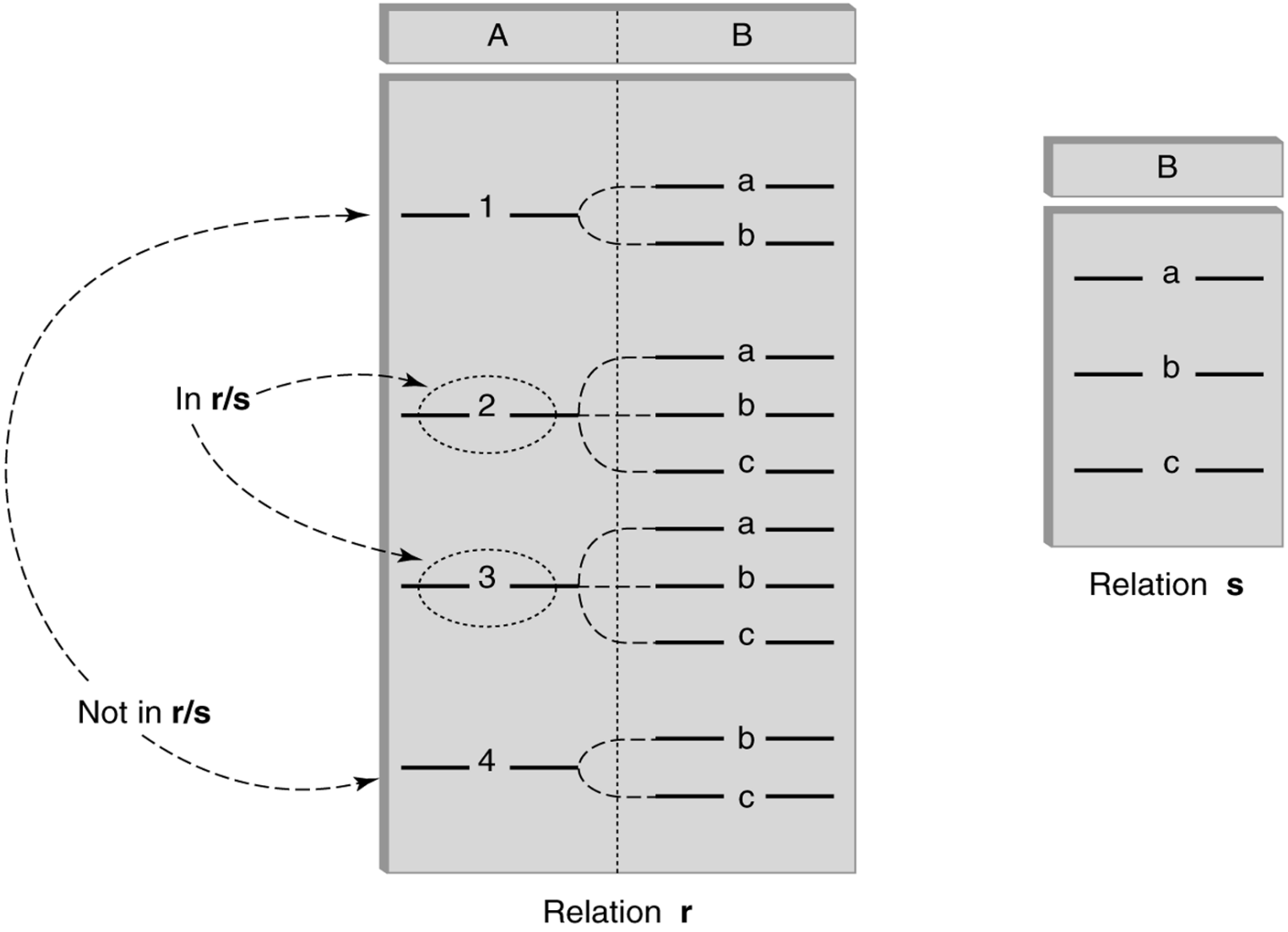
$U \underset{A<D \text{ AND } U.B \neq V.B}{\bowtie} V$

1	2	3	7	8	10

…Additional Operations

- **Assignment Operation**
  - The assignment operation denoted by ← is similar to assignment operation in programming that helps to assign the result of a relational algebra from the right into a relation variable to the left.
- **Division Operation**
  - The division operation denoted by ÷ or / is suited to queries that include "universal quantification" or the phrase "for all".
  - A division operation is applied to two relations $R(Z) \div S(X)$, where $X \subseteq Z$ and the result is $T(Y)$ where $Y = Z - X$.
    - $T1 \leftarrow \Pi_Y (R)$
    - $T2 \leftarrow \Pi_Y ((S \times T1) - R)$
    - $T \leftarrow T1 - T2$
  - **Note**: For tuples t to appear in the result T, the values in t must appear in R in combination with every tuple in S.

Division (cont'd)



Example:

- Consider the following relations R and S, then $R \div S$ is given as shown to the right.

R	
A	B
1	2
1	4
3	2
2	4
4	4
3	4

S
B
2
4

R÷S
A
1
3

…Example:

- Retrieve all the projects that “Jhon” and “Dave” are jointly working as programmers.

$$\Pi_{\text{Pr}ojects.Name}(\text{Pr}ojects \bowtie (\sigma_{Name=\text{"Pr}ogrammer\text{"}}(Teams \bowtie (EmpTeams \div \Pi_{EmpID}(\sigma_{Name=\text{"}Jhon\text{"}\ \text{OR}\ Name=\text{"}Dave\text{"}}(Employees))))))$$

Exercise:

s n o	p n o
s 1	p 1
s 1	p 2
s 1	p 3
s 1	p 4
s 2	p 1
s 2	p 2
s 3	p 2
s 4	p 2
s 4	p 4

A

pno
p2

B1

pno
p2
p4

B2

pno
p1
p2
p4

B3

A÷B1=?

A÷B2=?

A÷B3=?

Solution:

s n o	p n o
s 1	p 1
s 1	p 2
s 1	p 3
s 1	p 4
s 2	p 1
s 2	p 2
s 3	p 2
s 4	p 2
s 4	p 4

A

pno
p2

B1

sno
s1
s2
s3
s4

A÷B1

pno
p2
p4

B2

sno
s1
s4

A÷B2

pno
p1
p2
p4

B3

sno
s1

A÷B3

Extended Operations

- **Outer Join Operation**
  - **Left Outer Join**

    $R \mathbin{⟕} S$

    - Keeps every tuples in the left relation R
  - **Right Outer Join**

    $R \mathbin{⟖} S$

    - Keeps every tuples in the right relation S
  - **Full Outer Join**

    $R \mathbin{⟗} S$

    - Keeps every tuples in both relations R and S

Example:

- Consider the following relations R and S, then $R \ltimes S$ is given as shown to the right.

R

A	B
1	2
3	4
4	6

S

B	C	D
2	a	x
4	b	y
5	c	z
7	d	w

$R \ltimes S$

A	B	C	D
1	2	a	x
3	4	b	y
NULL	5	c	z
NULL	7	d	w

...Example:

- Write relational algebra that retrieves all the projects and the corresponding teams in the projects.
  - Projects(PrjId, PName, SDate, DDate, CDate)
  - Teams(TeamId, PrjId, TName, Descr)

$$\Pi_{\text{P.PName,T.TName}}\left(\left(\rho_P\left(\text{Pr}\,ojects\right) \;\overline{\bowtie}\; \rho_T\left(Teams\right)\right)\right)$$

...Extended Operations

- **Extended Projection Operation**
  - The **extended (or generalization) projection** extends the projection operation by allowing arithmetic functions to be used in the project list to compute and produce new columns.

Example:-Write a relational algebra for calculating the net pay of employees.

$$\Pi_{Name,\ Salary,\ \text{Salary-0.15*salary}\rightarrow\text{Tax},\ \text{Salary-Tax}\rightarrow\text{NetIncome}}\left(Employees\right)$$

...Extended Operations

- **Duplicate Elimination**
  - The duplicate elimination operator (δ) is used to eliminate duplicate tuples from the resulting relation.

$$\delta(R)$$

...Extended Operations

- **Aggregation and Grouping Operation**
  - Aggregation functions (operators) such as SUM, COUNT, MIN, MAX, and AVG are collection operators that return a single value as a result.
  - Aggregation operators are not relational algebra operators but they are used by the grouping operator (γ) that groups tuples according to their values in one or more attributes.

    $\gamma_L(R)$

    - L is either the **list of grouping attributes in order or list of aggregation functions** applied to the attributes of the relation R

Example:-Write a relational algebra that determines the number of teams all the employees are working in.

$$\gamma_{Name,\,\text{COUNT TeamId}}(Employees \bowtie EmpTeams)$$

Example: Aggregation

Relation R

A	B
1	3
3	4
3	2

SUM(A) = 7
COUNT(A) = 3
MAX(B) = 4
AVG(B) = 3

...Extended Operations

- **Sorting Operation**
  - **Sorting operator (τ)** turns a relation into a list of sorted tuples according to one or more attributes.
  - The operator is used as a final step since all the other operators work on either a set or a bag but not in a list.

    $\tau_S(R)$

    - S is a list of attributes of R indicating the sort order of the resulting relation.

Example:-Write a relational algebra that presents net pay of employees in order.

$$\tau_{Name}\left(\Pi_{Name,\ Salary,\ \text{Salary-0.15*salary}\rightarrow\text{Tax},\ \text{Salary-Tax}\rightarrow\text{NetIncome}}\left(Employees\right)\right)$$

Introduction to Relational Calculus

- The relational algebra is a procedural query language on a relational database model.
- A ***relational calculus*** ,however, is a *declarative and nonprocedural* expression that specifies a retrieval request.
- Hence, there is no description of how to evaluate the query in a relational calculus. Rather, a relational calculus expression specifies what to be retrieved.

Cont'd

- High-level, first-order logic description
  - A formal definition of what you want from the database

Example:-

– English:

"Find all sailors with a rating above 7"

– In Calculus:

$\{S \mid S \in Sailors \wedge S.rating > 7\}$

"From all that is, find me the set of things that are tuples in the Sailors relation and whose rating field is greater than 7."

Cont'd

- A relational calculus is classified into two as:
  - Tuple Relational Calculus, and
  - Domain Relational Calculus

Relational Calculus Building Blocks

- **Variables**

  TRC: Variables are bound to *tuples*.

  DRC: Variables are bound to *domain elements* (= column values)

- **Constants**

  7, "Foo", 3.14159, etc.

- **Comparison operators**

  =, ≤,≥, <, >, etc.

- **Logical connectives**

  ¬ - not

  ∧ – and

  ∨ - or

  ⇒ - implies

  ∈ - is a member of

- **Quantifiers**

  ∀X(p(X)): For every X, p(X) must be true

  ∃X(p(X)): There exists at least one X such that p(X) is true

sid	sname	rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

- **English example**: Find all sailors with a rating above 7
  - Tuple R.C.:

    $\{S \mid S \in Sailors \wedge S.rating > 7\}$

    "From all that is, find me the set of things that are tuples in the Sailors relation and whose rating field is greater than 7."
  - Domain R.C.:

    $\{\langle S,N,R,A\rangle \mid \langle S,N,R,A\rangle \in Sailors \wedge R > 7\}$

    "From all that is, find me column values S, N, R, and A, where S is an integer, N is a string, R is an integer, A is a floating point number, such that <S, N, R, A> is a tuple in the Sailors relation and R is greater than 7."

Tuple Relational Calculus

- A query in a tuple relational calculus (tuple calculus) is expressed as:

$$\{t \mid p(t)\}$$

  - Where t is a tuple variable and p(t) a predicate (condition) that is to be true for the tuple t.

- Formulas in the predicate of the tuple calculus are composed of atoms, variables and quantifiers $\exists$ (existential quantifier) and $\forall$ (universal quantifier)

Cont'd...

- ***Query*** has the form: {*T* | *p*(*T*)}

  p(T) denotes a formula in which tuple variable ***T*** appears.

- ***Answer*** is the set of all tuples *T* for which the *formula p*(*T*) evaluates to *true*.

- ***Formula*** is recursively defined:
  - start with simple *atomic formulas* (get tuples from relations or make comparisons of values)
  - build bigger and better formulas using the *logical connectives*.

Cont'd...

- *An Atomic formula is one of the following:*

  $R \in Rel$

  $R.a$ *op* $S.b$

  $R.a$ *op constant*

  op is one of $<, >, =, \leq, \geq, \neq$

- A *formula* can be:
  - an atomic formula
  - $\neg p,\ p \wedge q,\ p \vee q$ where p and q are formulas
  - $\exists R(p(R))$ where variable *R* is a tuple variable
  - $\forall R(p(R))$ where variable *R* is a tuple variable

Examples:

- *Find the sailor name, boat id, and reservation date for each reservation*

$$\{P|\exists R \in Reserves \exists S \in Sailors$$
$$(R.sid = S.sid \wedge$$
$$P.bid = R.bid \wedge P.day = R.day \wedge P.sname = S.sname)\}$$

sname	*bid*	*day*
Dustin	101	1996-10-10
Rusty	103	1996-11-12

…Examples:

- *Find the names of sailors who have reserved boat 103*

$$\{P|\exists R \in Reserves \exists S \in Sailors$$
$$(R.sid = S.sid \wedge R.bid = 103 \wedge$$
$$P.sname = S.sname)\}$$

sname	*bid*	*day*
Dustin	101	1996-10-10
Rusty	103	1996-11-12

…Examples:

- *Find the names of sailors who have reserved all boats*

$$\{P|\exists S \in Sailors \quad \forall B \in Boats$$
$$(\exists R \in Reserves(R.sid = S.sid \wedge B.bid = R.bid \wedge$$
$$P.sname = S.sname))\}$$

- "Find sailors S such that for all boats B there is a Reserves tuple showing that sailor S has reserved boat B"
- This query is equivalent to the division operator:

$$\rho(Tempsids, (\pi_{sid,bid}Reserves)/(\pi_{bid}Boats))$$

$$\pi_{sname}(Tempsids \bowtie Sailors)$$

…Examples:

a) List the names of all managers who earn more than £25,000.

{S.fName, S.lName | Staff(S) ∧
S.position = 'Manager' ∧ S.salary > 25000}

b) List the staff who manage properties for rent in Glasgow.

{S | Staff (S) ∧ $P ∈ PropertyForRent (P.staffNo =
S.staffNo) Ù P.city = 'Glasgow')}

…Examples:

- Find all the employees that are working on projects having a due date before January 1, 2007.

$$\left\{ e.Name \mid Employees(e)\ AND\ ((\exists et)(\exists t)(\exists p)(EmpTeams(et)\ AND\ et.EmpId = e.EmpId\ AND \right.$$
$$Teams(t)\ AND\ et.TeamId = t.TeamId\ AND$$
$$\left. \Pr ojects(p)\ AND\ t.\Pr jId = p.\Pr jId\ AND\ p.DDate < 1/1/2007\ )) \right\}$$

Domain Relational Calculus

- A query in a domain relational calculus (domain calculus) uses domain variables that take on values from an attributes domain rather than values for an entire tuple. It is expressed as:

$$\{< x_1, x_2, \cdots x_n >| \; p(x_1, x_2, \cdots x_n)\}$$

  - Where $x_1, x_2, \cdots x_n$ represent domain variables and P is the predicate as in the case of tuple calculus.
  - Formulas in the predicate are build in the same ways as the tuple calculus predicates.

Examples:

- Find the *loan-number*, *branch-name*, and *amount* for loans of over $1200

$$\{< l, b, a > \mid < l, b, a > \in loan \wedge a > 1200\}$$

- Find the names of all customers who have a loan of over $1200

$$\{< c > \mid \exists\, l, b, a\, (< c, l > \in borrower \wedge < l, b, a > \in loan \wedge a > 1200)\}$$

- Find the names of all customers who have a loan from the Perryridge branch and the loan amount:

$$\{< c, a > \mid \exists\, l\, (< c, l > \in borrower \wedge \exists b(< l, b, a > \in loan \wedge b = \text{“Perryridge”}))\}$$

or $\{< c, a > \mid \exists\, l\, (< c, l > \in borrower \wedge < l, \text{“Perryridge”}, a > \in loan)\}$

…Examples:

- Find the names of all customers having a loan, an account, or both at the Perryridge branch:

$$\{< c > \mid \exists\, l\, (\{< c, l > \in \textit{borrower} \wedge \exists\, b,a(< l, b, a > \in \textit{loan} \wedge b = \text{“Perryridge”})) \vee \exists\, a(< c, a > \in \textit{depositor} \wedge \exists\, b,n(< a, b, n > \in \textit{account} \wedge b = \text{“Perryridge”}))\}$$

- Find the names of all customers who have an account at all branches located in Brooklyn:

$$\{< c > \mid \exists\, s, n\, (< c, s, n > \in \text{customer}) \wedge \forall\, x,y,z(< x, y, z > \in \textit{branch} \wedge y = \text{“Brooklyn”}) \Rightarrow \exists\, a,b(< x, y, z > \in \textit{account} \wedge < c,a > \in \textit{depositor})\}$$

…Examples:

- Retrieve name and description of all the teams working on project named “banking db”.
  - Projects(<u>PrjId</u>, PName, SDate, DDate, CDate)
  - Teams(<u>TeamId</u>, PrjId, TName, Descr)

$$\{yz \mid (\exists x)(\exists a)(\exists b)(Teams(wxyz)\ AND\ \Pr ojects(abcde)\ AND\ b = "banking\ db"\ AND\ x = a)\}$$

THE END